

LAPORAN TUGAS
SINKRONISASI SISTEM OPERASI



Dosen Pengampu :

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

MOH. Ayatullah Mutawakki. F {2410651011}

Tanggal: 10-November-2025

PROGRAM STUDY TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

Contents

SOAL PEMROGRAMAN	2
A. Analisis Masalah:	2
B. Implementasi	2
KESIMPULAN.....	8
REFERENSI.....	9

SOAL PEMROGRAMAN

A. Analisis Masalah:

1. Pemahaman Soal

Didalam contoh soal terdapat 5 kamar saja dan costemernya 10 yang akan menggunakan kamar 5 tersebut. (10 costemer = 10 thread).

Nah disini muncul persoalan sinkronisasi ketika beberapa thread membaca struktur data Bersama, (misal: daftar kamar, sama jumlah kamar tersedia) secara bersamaan.

2. Identifikasi Masalah Sinkronisasi

Saat beberapa costumer booking bersamaan, bisa saja dua thread sama-sama melihat kamar yang kosong lalu keduanya langsung menempati kamar kosong itu. Nah, ini yang disebut race condition dibagian alokasi kamar.

Selain itu, kalau semua thread mencetak hasil ke layar secara bersamaan tanpa diatur, outputnya bisa campur aduk dan sulit untuk dibaca (yakni tumpah tindih tulisannya).

3. Desain Solusi

Disini saya memakai semaphore dari POSIX buat ngatur jumlah kamar yang bisa dipakai. Nilainya saya set awal 5, jadi Cuma ada 5 thread(costumer) aja yang bisa booking dan masuk ke bagian kritis “pesan dan tempati kamar” diwaktu yang sama.

Selain itu, saya pakai mutex buat:

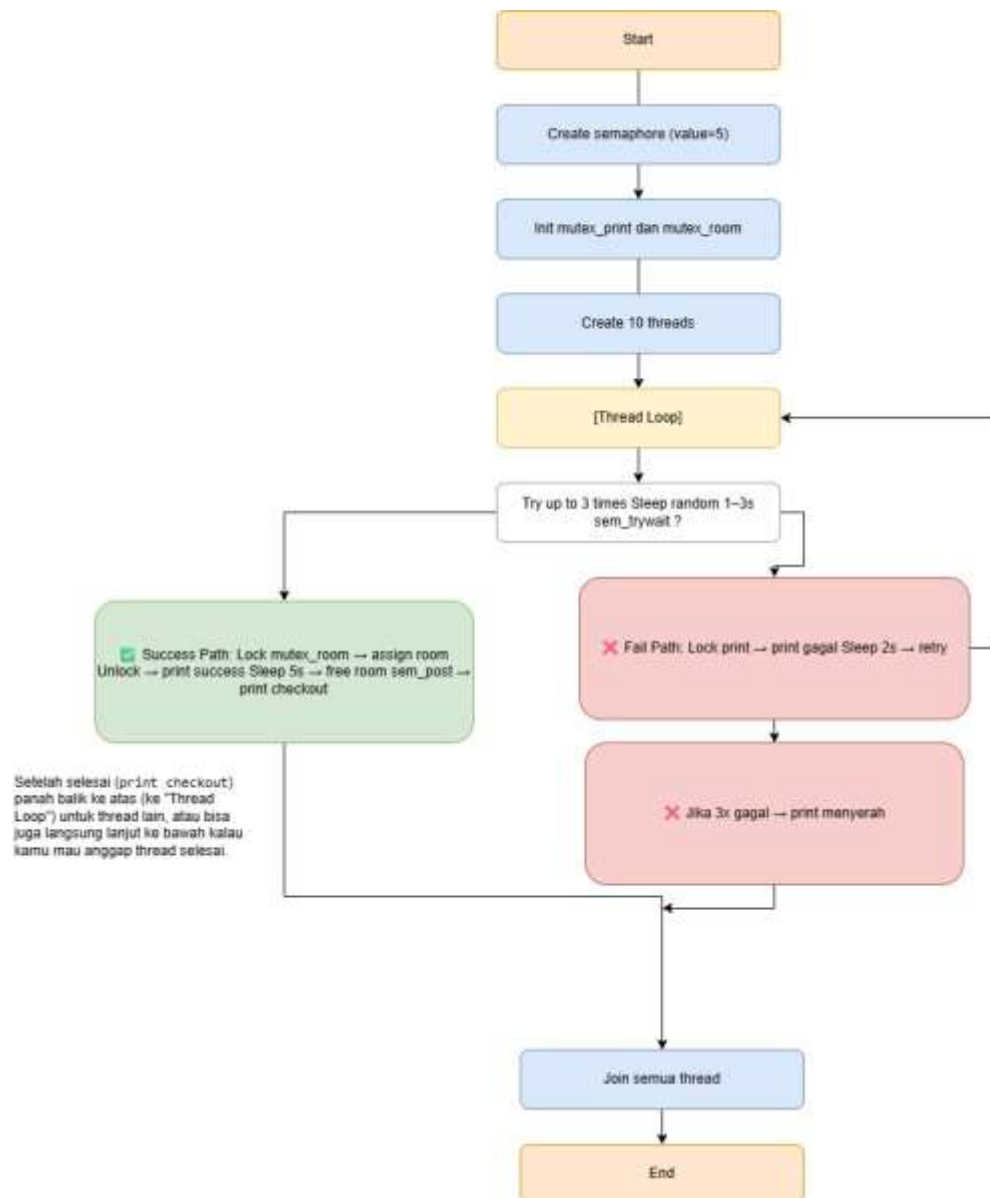
- Mutex_prtint: biar semua thread ngeprint ke layar, hasilnya tidak campur-campur dan tetap rapi
- Mutex_room: buat ngatur data kamar saat ada costumer yang memilih kamar kosong dan ngisi datanya, supaya tidak bentrok sama thread lain.

B. Implementasi

1. Penjelasan Algoritma

- Menggunakan pthread_create untuk membuat 10 thread.
- Setiap thread menjalankan customer_thread(void*arg) yang melakukan percobaan booking maksimal 5 kali.
- Untuk rondom delaynya menggunakan sleep / usleep dengan nilai acak dari 1 sampai 3 detik.
- Untuk timestamp output menggunakan strftime / localtime.

2. Flowchart/Diagram



3. Source Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

// Jumlah kamar maksimal yang bisa ditempati
#define MAX_ROOMS 5
```

```

// Jumlah thread yang mau booking
#define MAX_THREADS 10

// Variabel global
sem_t sem_kamar;           // semaphore buat batasin jumlah
kamar yang bisa dipakai pthread_mutex_t mutex_print; // buat
jaga biar output printf gak campur aduk pthread_mutex_t
mutex_room; // buat jaga data kamar waktu dicek/diganti int
kamar[MAX_ROOMS] = {0}; // 0 artinya kosong, 1 artinya
udah ditempati

// Fungsi yang dijalankan tiap thread (simulasi orang booking)
void* booking(void* arg) {
    int id = *(int*)arg; // ambil ID thread    int berhasil = 0; //
    buat cek apakah thread ini berhasil dapet kamar

    // Tiap thread nyoba booking sampe 3 kali    for (int attempt =
    1; attempt <= 3; attempt++) {    sleep(rand() % 3 + 1); // simulasi
    delay waktu proses booking (1-3 detik)

    // Coba masuk ke semaphore (cek apakah masih ada kamar
    kosong)
    if (sem_trywait(&sem_kamar) == 0) {
        // Kalau berhasil, berarti dapet jatah kamar
        pthread_mutex_lock(&mutex_room); // kunci akses kamar
        int i;
        for (i = 0; i < MAX_ROOMS; i++) {            if
        (kamar[i] == 0) { // cari kamar kosong
            kamar[i] = id; // isi kamar pake ID thread ini
            break;
        }
    }
    pthread_mutex_unlock(&mutex_room); // buka kunci
    akses kamar

    // Print hasilnya biar rapi
    pthread_mutex_lock(&mutex_print);            printf("
    Thread %d berhasil booking kamar %d
    (percobaan %d)\n", id, i + 1, attempt);
    pthread_mutex_unlock(&mutex_print);

    sleep(5); // simulasi nginep selama 5 detik

    // Waktunya checkout
    pthread_mutex_lock(&mutex_room);

```

```

        kamar[i] = 0; // kosongin kamar lagi
pthread_mutex_unlock(&mutex_room);

        sem_post(&sem_kamar); // balikin slot kamar ke semaphore (nambah 1)

        pthread_mutex_lock(&mutex_print);          printf(" Thread %d checkout
dari kamar %d\n", id, i +
1);
        pthread_mutex_unlock(&mutex_print);

        berhasil = 1; // tandain kalau berhasil booking          break;          //
keluar dari perulangan percobaan
    }          else {
        //          Kalau          gagal          (karena          kamar          penuh)
pthread_mutex_lock(&mutex_print);          printf(" Thread %d gagal booking
(percobaan %d)\n", id, attempt);
        pthread_mutex_unlock(&mutex_print);          sleep(2); // tunggu
2 detik sebelum nyoba lagi
    }
}

// Kalau udah 3x nyoba tapi tetap gagal    if (!berhasil) {
    pthread_mutex_lock(&mutex_print);          printf(" Thread %d menyerah
setelah 3x gagal.\n", id);          pthread_mutex_unlock(&mutex_print);
}

pthread_exit(NULL); // keluar dari thread
}

int main() {    srand(time(NULL)); // biar random sleep beda-beda tiap thread

    // Inisialisasi semaphore dan mutex
    sem_init(&sem_kamar, 0, MAX_ROOMS); // value awal semaphore = 5 (jumlah
kamar)          pthread_mutex_init(&mutex_print,          NULL);
pthread_mutex_init(&mutex_room, NULL);

    pthread_t threads[MAX_THREADS];    int
ids[MAX_THREADS];

    // Buat dan jalankan 10 thread (simulasi 10 orang booking)    for (int i = 0;
i < MAX_THREADS; i++) {        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, booking, &ids[i]);

```

```

    }

    // Tunggu semua thread selesai    for (int i = 0; i <
MAX_THREADS; i++) {

```

```

        pthread_join(threads[i], NULL);
    }

    // Bersihin resource
    pthread_mutex_destroy(&mutex_print);
    pthread_mutex_destroy(&mutex_room);
    sem_destroy(&sem_kamar);

    printf(" Semua thread selesai.\n");
    return 0;
}

```

4. Testing dan Output Cara menjalankan:

- Simpan kode file booking_hotel.c
- Buka terminal, komplikasi: gcc -pthread booking_hotel.c -o booking_hotel
- Jalankan:
./booking_hotel
- Outputnya:

```

aqil@DESKTOP-RGGJ3FG:~$ nano booking_hotel.c
aqil@DESKTOP-RGGJ3FG:~$ gcc -pthread booking_hotel.c
./booking_hotel
● Thread 1 berhasil booking kamar 1 (percobaan 1)
● Thread 4 berhasil booking kamar 2 (percobaan 1)
● Thread 5 berhasil booking kamar 3 (percobaan 1)
● Thread 6 berhasil booking kamar 4 (percobaan 1)
● Thread 3 berhasil booking kamar 5 (percobaan 1)
✗ Thread 9 gagal booking (percobaan 1)
✗ Thread 2 gagal booking (percobaan 1)
✗ Thread 7 gagal booking (percobaan 1)
✗ Thread 8 gagal booking (percobaan 1)
✗ Thread 10 gagal booking (percobaan 1)
● Thread 1 checkout dari kamar 1
● Thread 5 checkout dari kamar 3
● Thread 4 checkout dari kamar 2
● Thread 6 checkout dari kamar 4
● Thread 10 berhasil booking kamar 1 (percobaan 2)
● Thread 3 checkout dari kamar 5
● Thread 9 berhasil booking kamar 2 (percobaan 2)
● Thread 8 berhasil booking kamar 3 (percobaan 2)
● Thread 2 berhasil booking kamar 4 (percobaan 2)
● Thread 7 berhasil booking kamar 5 (percobaan 2)
● Thread 10 checkout dari kamar 1
● Thread 9 checkout dari kamar 2
● Thread 8 checkout dari kamar 3
● Thread 2 checkout dari kamar 4
● Thread 7 checkout dari kamar 5
❖ Semua thread selesai.

```

5. Analisis Hasil

Dari beberapa percobaan yang aku lakukan, program ini berjalan sesuai ekspektasi. Saat jumlah kamar masih tersedia, customer langsung bisa booking tanpa masalah. Tapi kalau kamar udah penuh, customer lain harus nunggu atau gagal dan nyoba lagi sampai maksimal 3 kali.

Penggunaan semaphore terbukti berhasil membatasi jumlah kamar yang bisa dipakai secara bersamaan (maksimal 5), dan mutex juga efektif supaya output printf gak berantakan serta data kamar gak saling rebutan antar thread.

Untuk performa program Selama dicoba, program jalan lancar walaupun ada 10 thread yang aktif barengan. Karena ini cuma simulasi, beban CPU juga gak berat. Sistem sinkronisasi yang aku pakai bikin program tetap stabil dan gak ada error atau crash pas dijalankan.

Pelajaran yang saya dapat dari tugas ini saya makin paham bagaimana cara kerja sinkronisasi antar thread pakai mutex dan semaphore. Saya juga ngerti kenapa penting banget mencegah race condition, biar hasil program tetap konsisten.

KESIMPULAN

Dari hasil percobaan, bisa disimpulkan kalau sitem booking hotel ini udah berjalan dengan baik dan sesuai dengan logic soal.

Penggunaan semaohore sangat membantu untuk mengatur jumlah kamar yang bisa dipakai bersamaan, sementara mutex menjaga supaya data kamar dan tampilan output gak rusak karena diakses bersamaan oleh thread.

Tantangannya:

Bagian paling susah itu waktu ngatur biar gak ada thread yang booking kamar yang sama. Tapi setelah ngerti cara kerja pthread_mutex_lock dan sem_trywait, jadi lebih jelas alurnya.

Saran pengembangan:

Program ini bisa dikembangkan lagi misalnya dengan menambahkan waktu check-in & check-out yang berbeda, sistem prioritas customer, atau interface GUI sederhana biar lebih menarik dan kelihatan kayak aplikasi beneran.

REFERENSI

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th Edition). Wiley.
- Dokumentasi POSIX Threads:
<https://man7.org/linux/manpages/man7/pthreads.7.html>
- Tutorial Semaphore & Mutex C: <https://www.geeksforgeeks.org/semaphores-inprocess-synchronization/>
- Dokumentasi sem_t: https://man7.org/linux/man-pages/man3/sem_init.3.html